2 - Pointers → Array

T-1 : Start & end → start++ , end-- → * Sorted array/string * Pair, triplets * Partitioning

T-2 : Slow & fast pointers → int j=0 for(int i=0 ton) if(cond) {j++}; → * find middle * Cycle detection in Linked list or number seq.

Trick → * Sorted Array * Have to find pairs (not group like subarray)

Example → # Leet-167 Two Sum II → Start & end pointers

* Array is Sorted
* T- O(n)
* if (sum > target) end--;
* if (sum < target) start++;
* Can also solve using Hashmap → T- O(log n)

In Leet-1 Two Sum → Hash map → T- O(log n)

* Array was not sorted so, need to sort first before using 2-pointer → T- O(n log n)

Leet-977 Square of Sorted Array → Start-End pointer + 1 extra pointer to iterate in answer storing array.

* Array is Sorted [-4,-1,0,3,10]
* int s=0 e=n-1 pos=n-1 → 3 pointers
* if(abs(num[s]) > abs(num[e]) ans[pos] = num[s] * num[s] s++
  else ans[pos] = num[e] * num[e] e--
  pos--;

filling ans vector from back side (left ← right)

abs(-4) < abs(10) → 100 So, store and --;

abs(-4) > abs(3) → 16 So store s++;

pos--; → moves right to left

T- O(n) S- O(n) → storing ans in vector

2nd method → * store sq in same array by replacing element with their squares * then sort. array * T- O(n log n) S- O(1)

Alanta Officer

Leet-26 Remove Duplicates from Sorted Array → Slow & fast pointer

* Can't use extra vector to store Answer
* Have to make updates in original array

```
int j = 0
int i = 0 to n
if (num[i] != num[j])
    i++
    num[i] = nums[j];
```

Leet-15 3-SUM → 3 pointers (start, end, ...)

* Brute → * 3 nested loops T = O(n³) S = O(2 × no of pairs)
  * Insert in SET T = O(log(no of pairs))
  Overall → T = O(n³ × log(no of pairs))

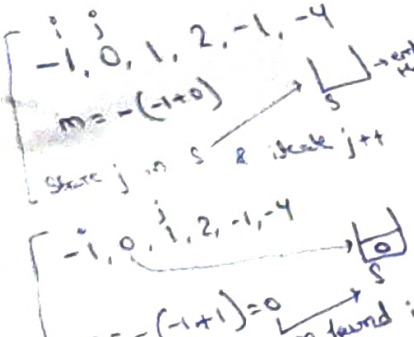
* use set to store unique pairs
* 3 loops to find pair with sum = 0
* Sort elements of pairs before storing into set
* this way only unique pairs will go into set

```
Set<vector<int>> st → set of vectors
for(i=0 to n)
  for(k=i+1 to n-1)
    for(j=k+1 to n)
      if(n[i]+n[j]+n[k]==0)
        vector<int> temp = {n[i], n[k], n[j]}
```

Store each found pair in vector ←
Sort the elements of pairs stored in vector ←
only unique pairs will go into set ← `st.insert(temp)`

```
sort(temp.begin(), temp.end())
return vector<vector<int>>(st.begin(), st.end())
```

* Better → * 2 nested loops T = O(n²)
  * Hashset T = O(log(no of pair))
  Overall → T = O(n² × log(no of pair))



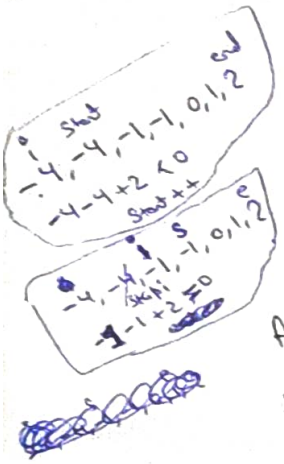

```
Set<vector<int>> st
for(i=0 to n)
  Set<int> S → Hash set
  for(j=i+1 to n)
    int m = 0 - (n[i] + n[j]) → target
    if(S.find(m) != S.end())
      vector<int> v = {n[i], m, n[j]}
      sort(v.begin(), v.end())
      st.insert(v);
    S.insert(n[j]) → store j into set to quick look of m & then move j ahead
return vector<vector<int>>(st.begin(), st.end());
```

* Optimal ⟶ * Fix 1 pointer & iterate in loop
* *Start & *End pointer
* T— $O(N \log N) + O(N^2)$
  - $O(N \log N)$ → Sort
  - $O(N^2)$ → for each i (loop) we are serachy j & K → end / Start → so nested loop
* S— O(no of pairs)

* Sort the array first
* for (i=0 to n) if(i>0 && num[i] == num[i-1]) Continue;
  - ↳ except for 1st element, it will check for rest all elements that if n[i] == n[i-1] same as prev elem then skip & move to next i

```
int j = i+1;
int K = n-1;
while (j<K)
    int S = n[i] + n[j] + n[K]
    if (S>0)  K--;
    else if (S<0) j++;
    else{ vector<int> temp = {n[i], n[j], n[K]};
          ans.push_back(temp);
          j++;
          K--;
          while (j<K && n[j] == n[j-1]) j++;
          while (j<K && n[K] == n[K+1]) K--;
    }
```

Found S=0 ← else{
Store pairs in vector
Store into ans vector
skip duplicates j & K

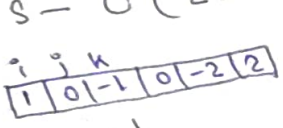


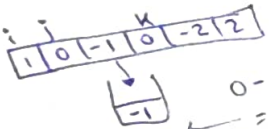
Leet-18 4Sum (Similar to 3 Sum)

* Brute ⟶ * 4 nested loops
  - T— $O(N^4)$
  - S— O(2 × no of pairs)

* Better ⟶ * 3 nested loops + Hashset
  - T— $O(n^3 \times \log(m))$
  - S— $O(2 \times m) + O(N)$
  - m → no of pairs

take long long rem = target - (1LL * n[i] + n[j] + n[k]) → int





0-(1+0+0) = -1 found in Hashset
So store 1,0,-1,0 in pair
Sort → -1,0,0,1 store in set

* optimal ⟶ * 2 nested loops + 2 end pointers (start, end)
  - * Sort the array first the fix i & j in nested loops
  - * for every i & j start++, end--
  - * long long Sum = 1LL * n[i] + n[j] + n[start] + n[end];
  - T— $O(N^3)$
  - S— O(no of Pair)

3 Sum Smaller — Algo Master - 25

* Same Concept as optimal Solution of 3Sum
* T— $O(n^2)$ S- $O(1)$
* Sort (n.begin(), n.end())

```
int n = nums.size();
int Count = 0;
for (i = 0 to n-2)
    int left = i+1
    int right = n-1;
    while (left < right)
        int sum = n[i] + n[left] + n[right]
        if (sum < target)
            Count += (right - left)   → all elements from left+1 to right will have sum < target
            left++;
        }
        else{
            right--;
        }
```

Can't use Hashmap

Leet- 16 3 Sum Closest

* Same Concept as optimal Soln of 3 Sum
* T— $O(n^2)$ S- $O(1)$
* int ClosestSum = n[0] + n[1] + n[2]

```
Sort(n.begin(), n.end())
!
if(abs(target - sum) < abs(target - closestSum)
    closestSum = sum;

if(s > target) end--
if(s < target) s++
if (s = target) return sum;
```

Leet- 75 Sort Colors

Dutch flag (3-pointers)

* 3 pointer → low = 0 mid = 0 high = n-1 (↳ r)

```
while (mid <= r)
    if (n[mid] == 2)
        Swap (n[mid], n[r])
        r--;
    else if (n[mid] == 0)
        Swap (n[mid], n[l]);
        l++;
        mid++;
    else
        mid++;
```

low = 0
mid = 0
High = n-1

T- $O(n)$
S- $O(n)$

other method → Sorting T- $(n \log n)$